



Week-2

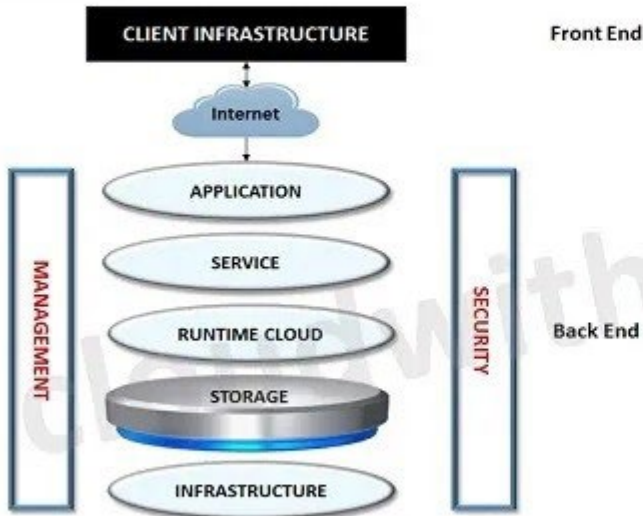
Cloud Computing Concepts



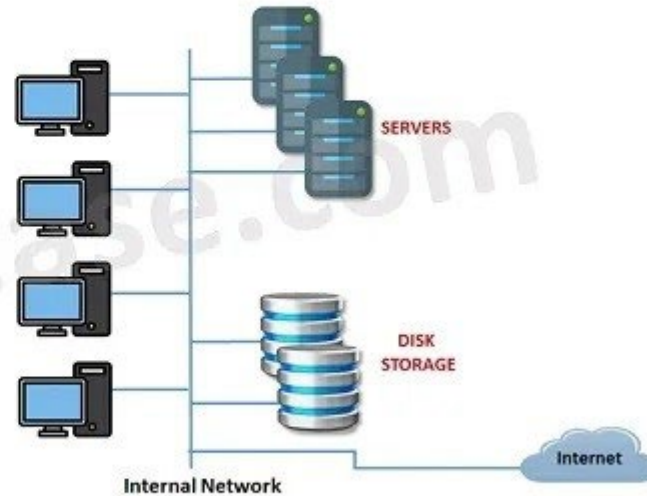
In Systems Design, one of the key factor to decide is to either go with traditional server-based architecture or Cloud.



Cloud Computing vs Traditional Computing



Cloud Computing



Traditional Computing



THERE IS NO CLOUD
IT'S JUST SOMEONE ELSE'S COMPUTER



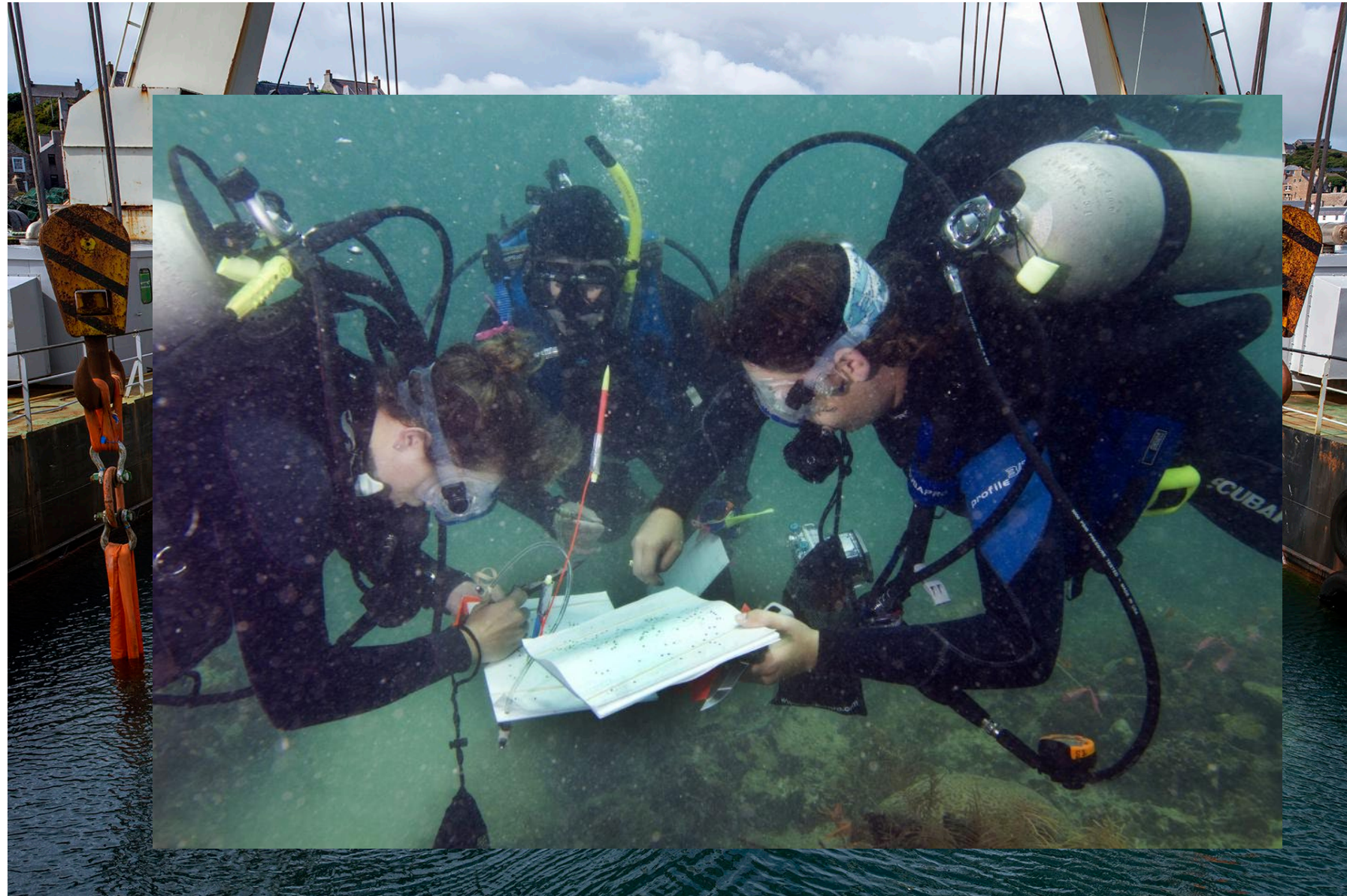
50 regions worldwide **140** available in 140 countries



- Available region
- Announced region
- Availability Zones present

Microsoft's [Project Natick](#) team deployed the Northern Isles datacenter [Scotland's Orkney Islands](#) 117 feet deep to the seafloor in spring 2018.

For the next two years, team members tested and monitored the performance and reliability of the datacenter's servers.



50 regions worldwide 140 available in 140 countries

Why so many locations?



Scale.

Latency.

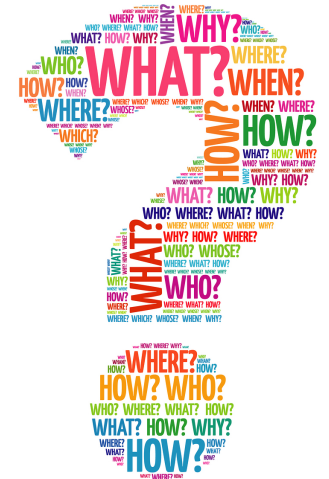
Jurisdiction/Legal issues.

Compliance.

Government snooping.

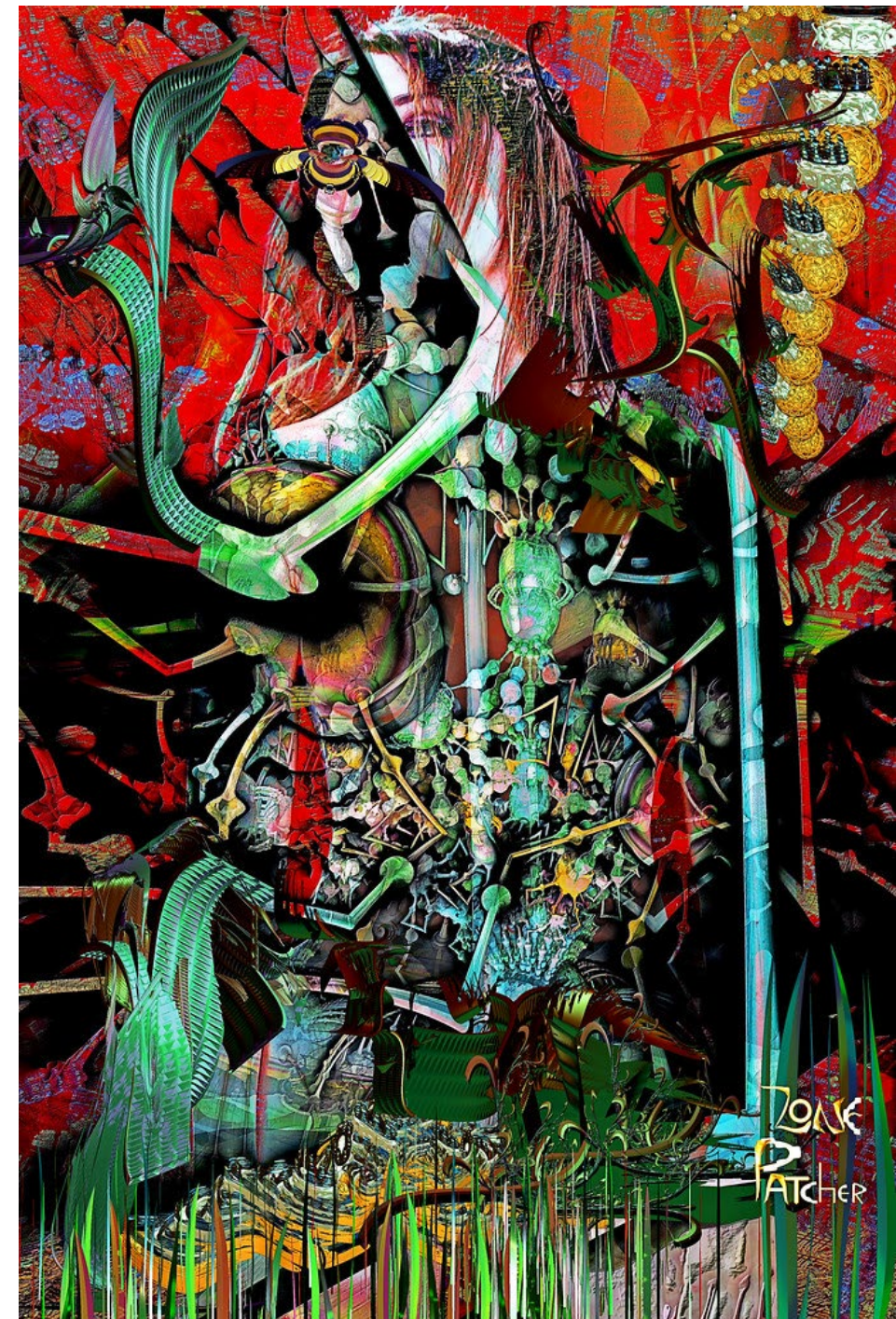
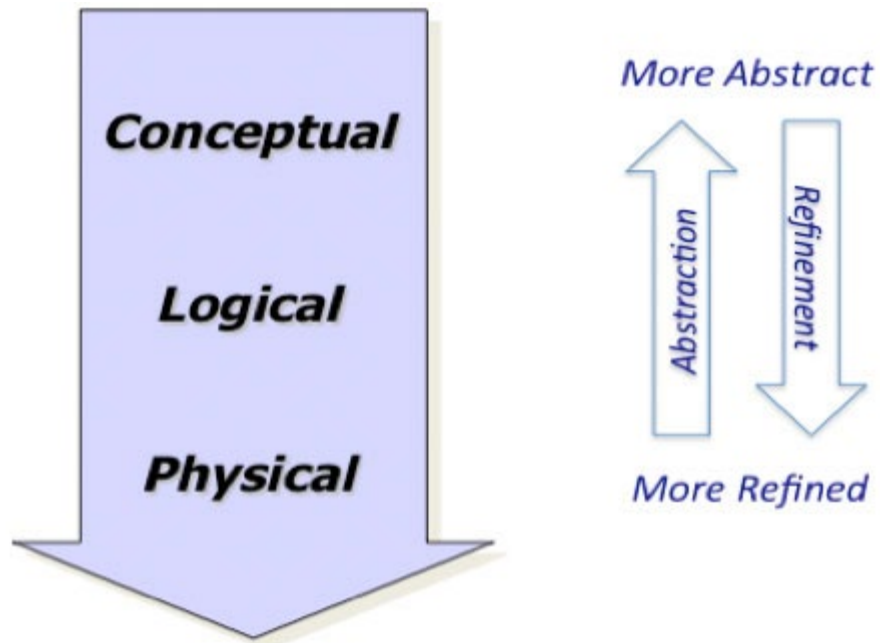
- Available region
- Announced region
- Availability Zones present

How is it possible to leverage these many thousand servers through abstraction?



Conceptual Abstraction

A representation that hides specifics from a consumer of services



Cloud Building Blocks

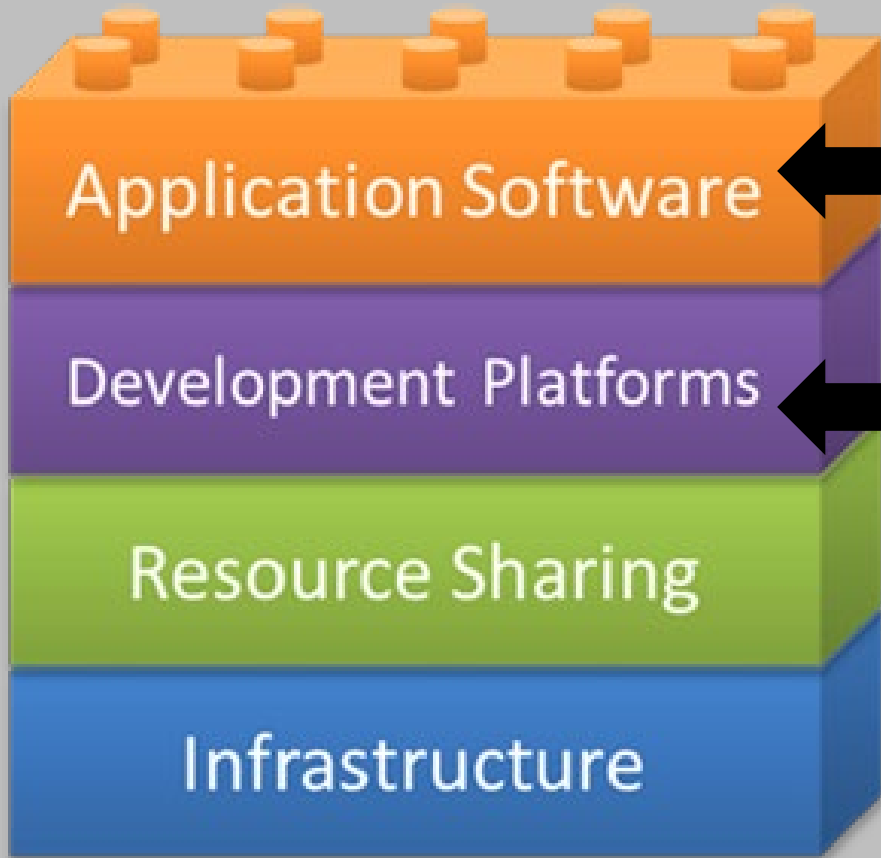
The Cloud has a variety of build blocks which may be subsequently used to realise a variety of paradigms/abstractions.

These are:

- *Application software*
- *Development Platforms*
- *Resource Sharing*
- *Infrastructure*



Cloud Building Blocks



Application software:

The top layer in the stack is the application software, which normally is the system component that the end user utilizes.

Development platforms:

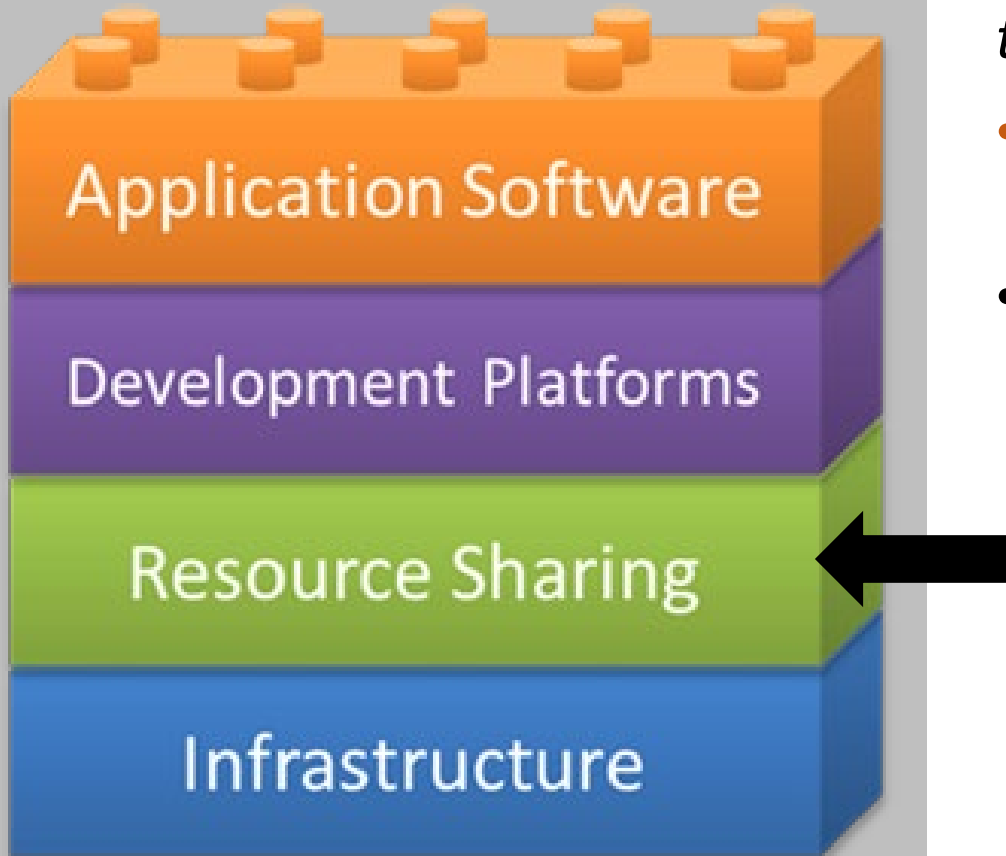
The next layer, development platforms, allows application developers to write application software in terms of a cloud's application programming interface (API).

*Development platforms typically provide specifications that developers can use for **routines, data structures, object classes, libraries, and variables.***

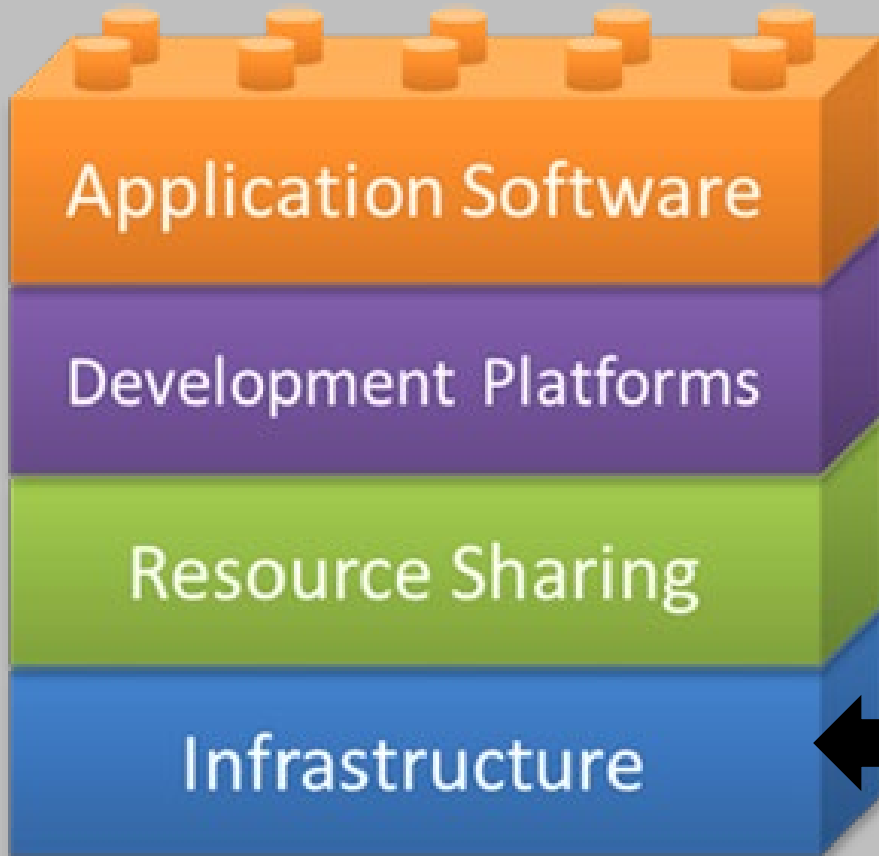
Cloud Building Blocks

Resource sharing: Resource sharing mechanisms, the third layer, embody some *key cloud ideas*:

- *Provide software, computation, network and storage services.*
- *Allow a **shared environment** whereby multiple hardware images (for example, virtual machines) and system images (for example, general-purpose OSs) can run side by side on a single infrastructure along with security, resource, and failure isolations. These isolation properties are provided by a combination of hardware and software techniques that are covered later.*
- *Consolidate physical servers into virtual servers that run on fewer physical servers.*
- *Deliver agility and elasticity to rapidly respond to users' resource and service demands.*



Cloud Building Blocks



Infrastructure: Physical resources compose the bottom layer and are primarily deployed on the cloud provider's side. The broad resource classes include the following:

- *Compute resources*, typically servers, which are computers designed for enterprise computing (as opposed to user workstations). They usually are rack mounted to utilize space efficiently.
- *Storage resources* that maintain the cloud's data. Application storage usually is charged in terms of capacity usage (for example, per gigabyte or terabyte).
- *Network resources* that enable communication between servers as well as between servers and clients.
- *Software* that manages the compute, network, and storage infrastructure.

In a *Cloud* context, these vast array of servers are typically abstracted using **aaS paradigms*. The most common paradigms are:

i. Software as a Service (SaaS)

*any application in which the end user has access to a **software application over the network** and pays based on a variety of business models, some of which are free.*

ii. Platform as a Service (PaaS)

*is the offering of **software development platforms** as a service, which are utilized to develop SaaS applications*

iii. Infrastructure as a Service (IaaS)

*is the leasing of **virtualized infrastructure over the network**. In this last model, the end user has the flexibility to install and use any software they please on the leased infrastructure.*

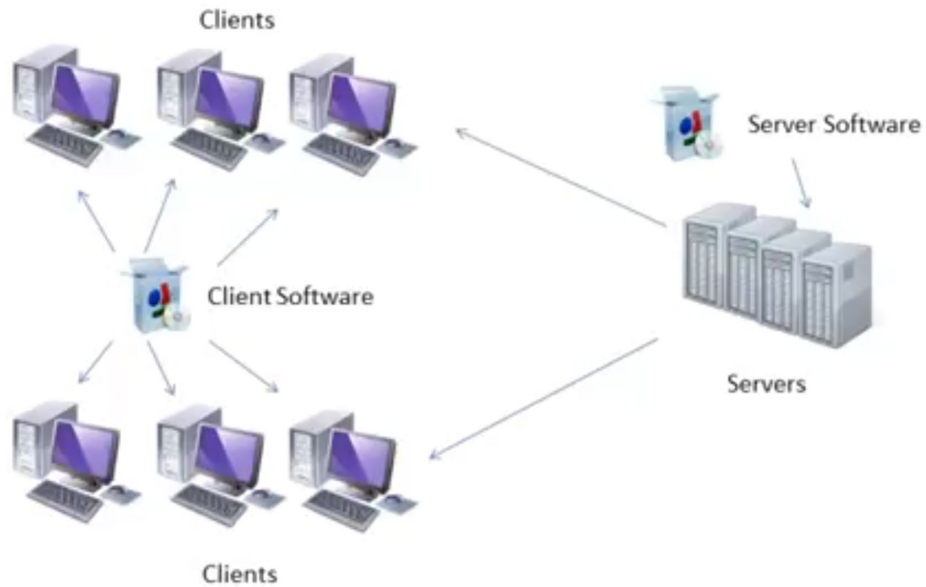


Software as a Service (SaaS)

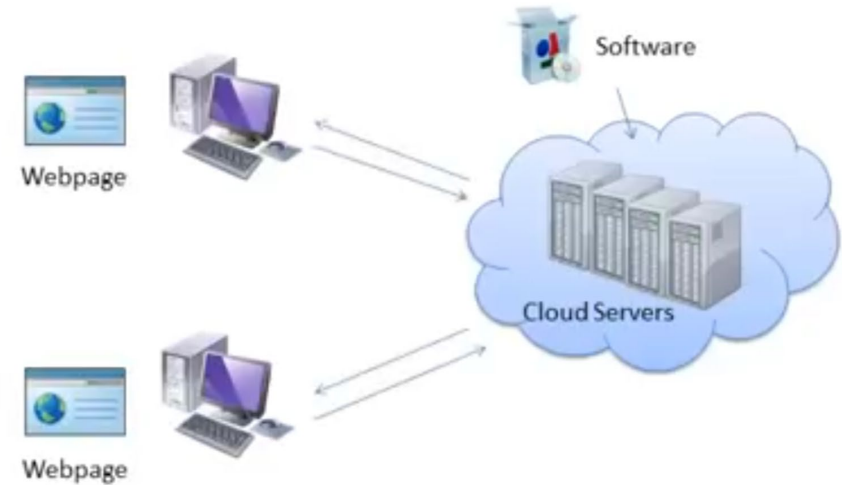
compared with

Traditional Software

Traditional Software



Software as a Service





Software as a Service (SaaS)

*Software as a service (SaaS) is a software delivery model in which software and associated data are hosted on a cloud. SaaS applications typically are accessed by users using a **thin client** via a **web browser**.*

SaaS is one of the **most common** cloud service models in which the cloud provider delivers software as an internet service.

SaaS users simply use their browsers to access the software, thus eliminating the need to **install, run, and maintain** (*update, patch, reconfigure, and so on*) the application on their computers.



Software as a Service (SaaS)

*SaaS has become a common software delivery model for **many business applications**, including accounting, collaboration, customer relationship management (**CRM**), management information systems (**MIS**), enterprise resource planning (**ERP**), invoicing, human resource management (**HRM**), content management (**CM**), and service desk management.*

*With SaaS, **the provider maintains** the software and the **required infrastructure** to run it.*

*The provider routinely **develops** the software, and enhancements are automatically made available the next time a user logs on to the service. In addition, any application data that results from the use of the service resides on the cloud and is available to the user from any location.*



Software as a Service (SaaS)

SaaS characteristics

*A vast majority of SaaS solutions are based on what is referred to as **multitenant architecture**.*

*In this architecture, a single version of the application, with a single configuration, is used for every **customer** (referred to as a **tenant**).*

To enable the service to scale well, it might be installed on several servers at the provider's side.

***Dynamic scaling** is utilized to allow more users to use the service as it becomes more popular.*



Software as a Service (SaaS)

Typical characteristics of SaaS include:

- Access to the software service is **web based***
- Software is **managed from a central location** by the cloud provider.
- Software is delivered in a **one-to-many model** in which "one" is the cloud provider and "many" is the cloud users.
- The cloud **provider handles** software upgrades and patches.



Software as a Service (SaaS)

SaaS pricing models

*Unlike traditional software, which is sold under the software licensing model (with an upfront license cost and an optional ongoing support fee), SaaS providers generally price applications using a **monthly or annual subscription fee**.*

*This model enables SaaS to fulfill one of the main purported advantages of cloud computing - **reducing the capital expenditure** or the upfront cost of software. SaaS providers typically charge based on usage parameters, such as the number of users using the application.*



Software as a Service (SaaS)

SaaS use cases

SaaS is a good model for certain types of applications, such as:

- *Applications that are fairly standardized and do not require custom solutions. **E-mail** is a good example of a fairly standardized application.*
- *Applications that have a **significant need for remote/web/mobile access**, such as mobile sales management software.*
- *Applications that have a **short-term need**, such as collaborative software for a particular project.*
- *Applications in which **demand spikes significantly**, such as tax or billing software that is used once a month.*



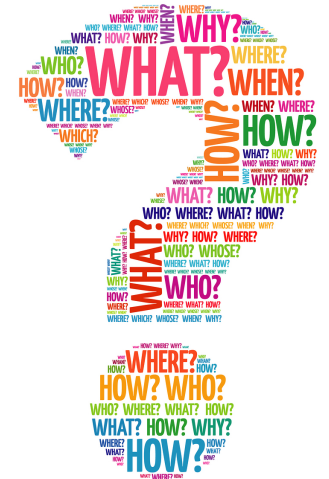
Software as a Service (SaaS)

SaaS use cases

*However, there are situations where SaaS **may not be the right choice**, such as:*

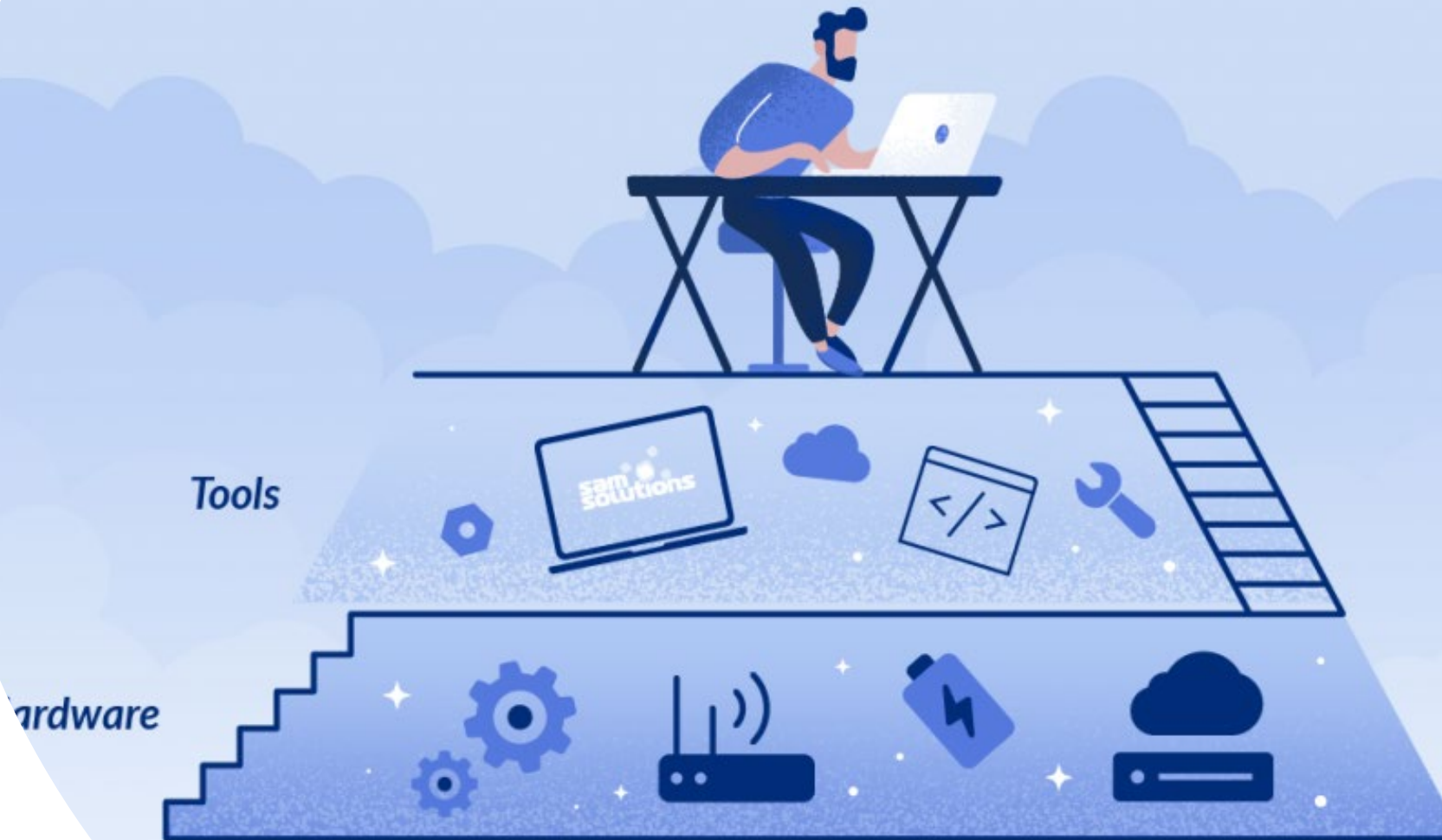
- *Applications that require offline access to data*.*
- *Applications that require significant customisation.*
- *Applications in which policies or regulations disallow data from being hosted externally.*
- *Applications in which existing in-house solutions satisfy all of the organization's needs.*

Can you name a SaaS based service?





Platform as a Service





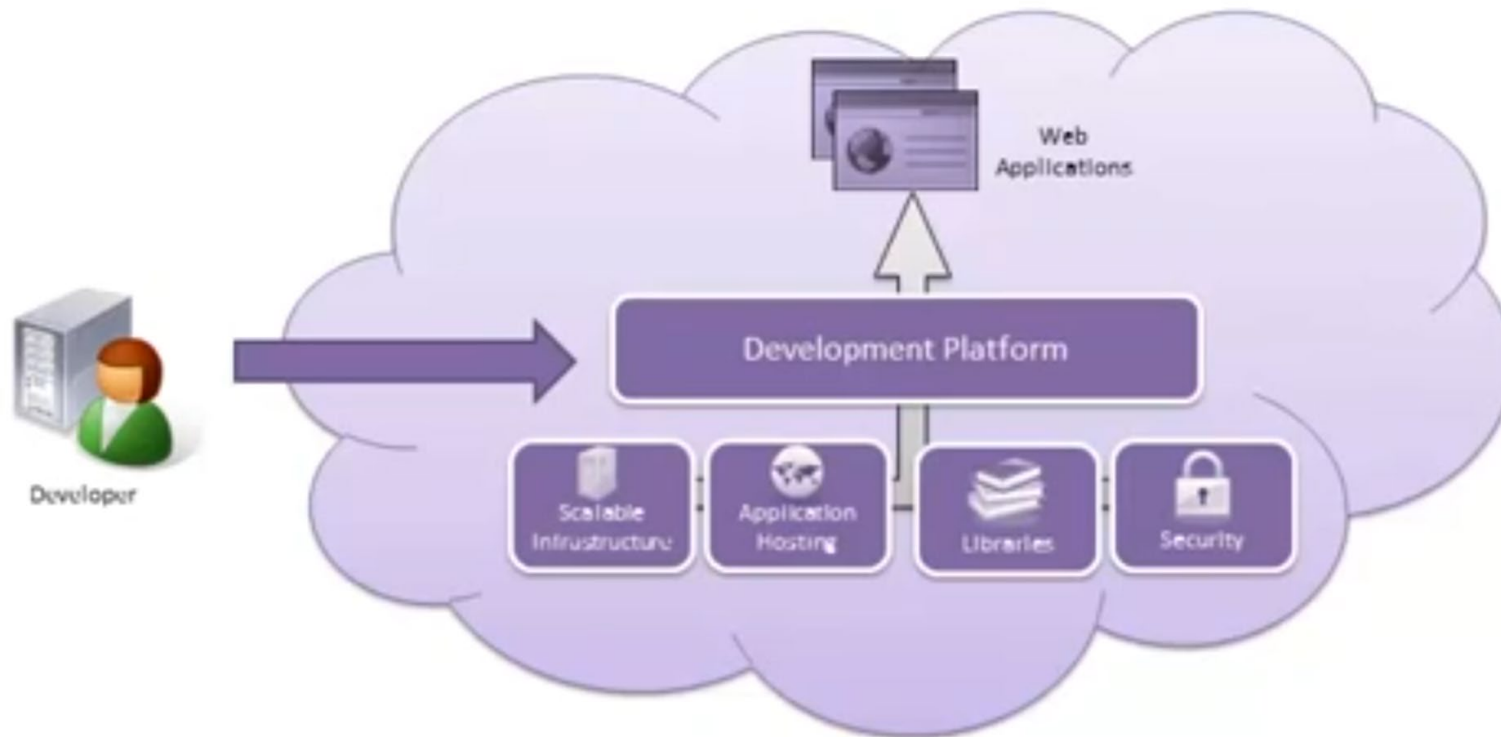
Platform as a service (PaaS)

*Platform as a service (PaaS) is a computing paradigm that allows for the **creation of web applications in a simplified manner** without the complexity of purchasing and maintaining any of the underlying software and infrastructure.*

PaaS-based offerings allow users to develop, deploy, and scale applications on platforms that are offered by cloud providers. PaaS is analogous to SaaS, except that rather than software delivered over the web, it is a platform for the creation of software that is delivered over the web.

Platform as a Service (PaaS)

Platform as a Service





Platform as a service (PaaS)

PaaS characteristics

PaaS offerings vary among providers but usually feature some basic functionality, which includes:

- *Services to develop, test, deploy, host, and maintain applications in the same integrated development environment (IDE).*
- *Web-based user interface (UI) creation tools to help create, modify, and test various UI scenarios.*
- *Multitenant architecture in which multiple concurrent users utilize the same development tools.*
- *Built-in scaling mechanisms of deployed software that can be handled automatically by the cloud provider by load-balancing and failover mechanisms.*



Platform as a service (PaaS)

PaaS pricing models

*Unlike the SaaS pricing model (which is a subscription- or advertisement-based model), **PaaS usually is priced in terms of usage of the platform.***

*For example, **Azure Functions' pricing model charges for total number of executions as well as execution time.***

Consequently, the more an application developed using PaaS gets used, the more the PaaS user gets charged.



Platform as a service (PaaS)

PaaS use cases

PaaS is a good model for certain types of applications, such as:

- *Rapid application development scenarios.*
- *Applications that require web-based infrastructure to handle varying loads from users.*
- *Applications that may not need redeployment or migration to other platforms in the future.*



Platform as a service (PaaS)

PaaS use cases

*There are certain scenarios in which PaaS **may not be ideal**, such as:*

- *When the **application needs to be highly portable** in terms of where it is hosted because PaaS APIs can vary from one PaaS provider to another.*
- *When **proprietary languages or APIs could impact** the development process or cause trouble in the future due to vendor lock-in.*
- *When **application performance requires customization** of the underlying hardware and software.*

Platform as a service (PaaS)

PaaS Examples

- AWS Elastic Beanstalk.
- Windows Azure.
- Heroku.
- Force.com.
- Google App Engine.
- OpenShift.



Google App Engine





IaaS

Infrastructure as a service (IaaS)

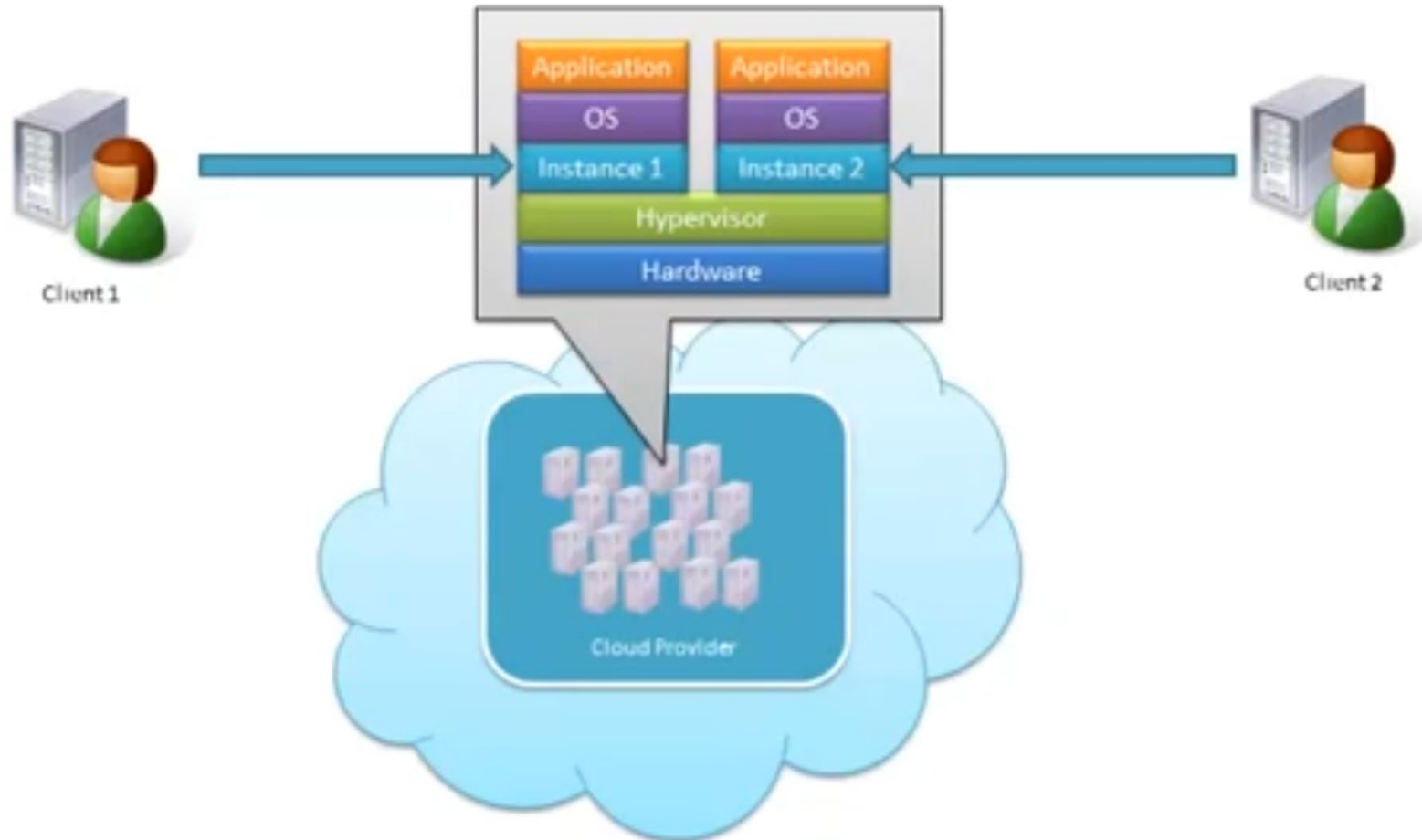
*Infrastructure as a service (IaaS) is a cloud computing model in which cloud providers make computing resources available to clients, usually in the form of **instances or virtual machines**.*

*In the IaaS model, providers **rent out** compute resources in the form of **instances or virtual machines**, which have some form of **configurable CPU, memory, disk, and network bandwidth** attached to them.*

After provisioning, IaaS users can remotely connect to these instances and configure their choice of platforms and applications. This model offers the most flexibility to the IaaS users in terms of software development and deployment. Rather than purchasing servers, software, datacenter space, or network equipment, users rent those resources as a fully outsourced service on demand.

Infrastructure as a service (IaaS)

Infrastructure as a Service

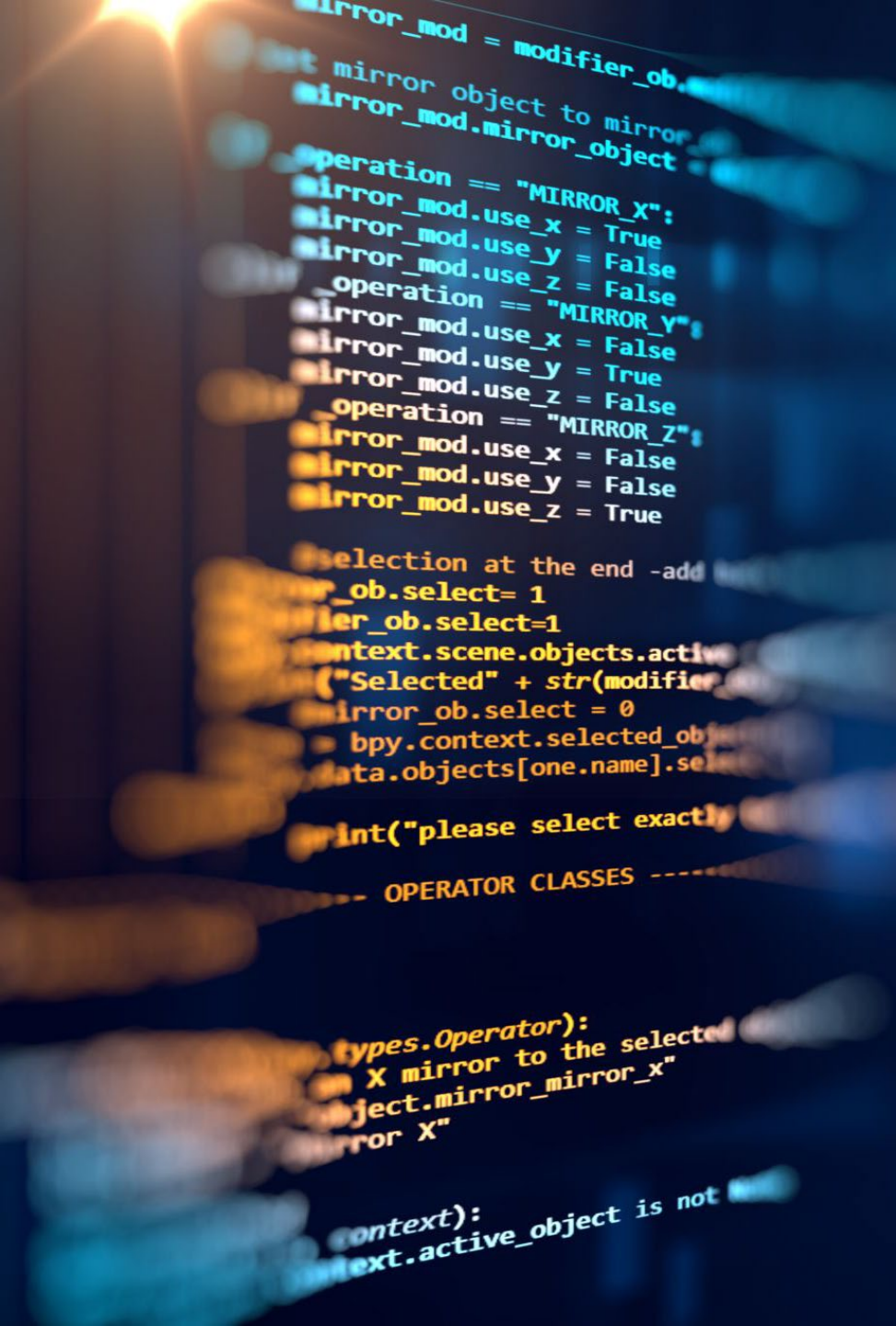


Infrastructure as a service (IaaS)

IaaS characteristics

IaaS has the following characteristics:

- **Computing resources** are provided to IaaS users as a service.
- IaaS providers provide tools that enable IaaS users to configure the dynamic scaling of resources.
- IaaS providers usually have different resource offerings at different costs and follow a utility pricing model (typically calculated hourly).
- The same physical resources are shared among multiple users.



Infrastructure as a service (IaaS)

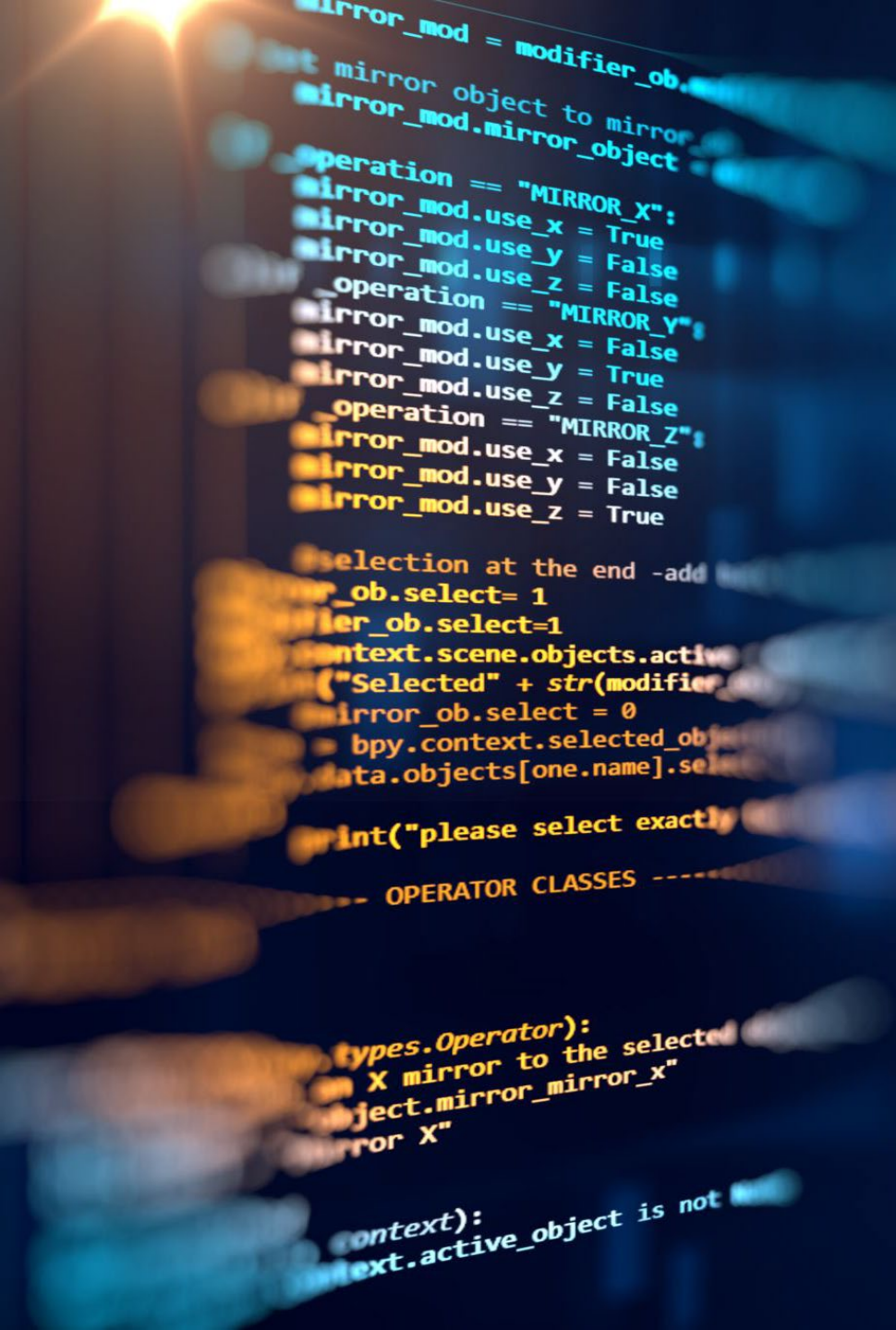
IaaS pricing models

Unlike the SaaS pricing model or the PaaS model, IaaS *usually is priced on an hourly basis, per instance*. For example, Azure Virtual Machines offer a spectrum of compute resources as virtualized OS instances, which vary in compute, memory, storage, and bandwidth.

Cloud providers can also choose to bill on a **prorated** or non-prorated basis.

On a **prorated** basis, each partial hour is billed partially, while on a non-prorated basis, each partial hour is billed as a full hour.

This difference becomes significant when IaaS users need a large number of instances for a short period of time for burst processing.

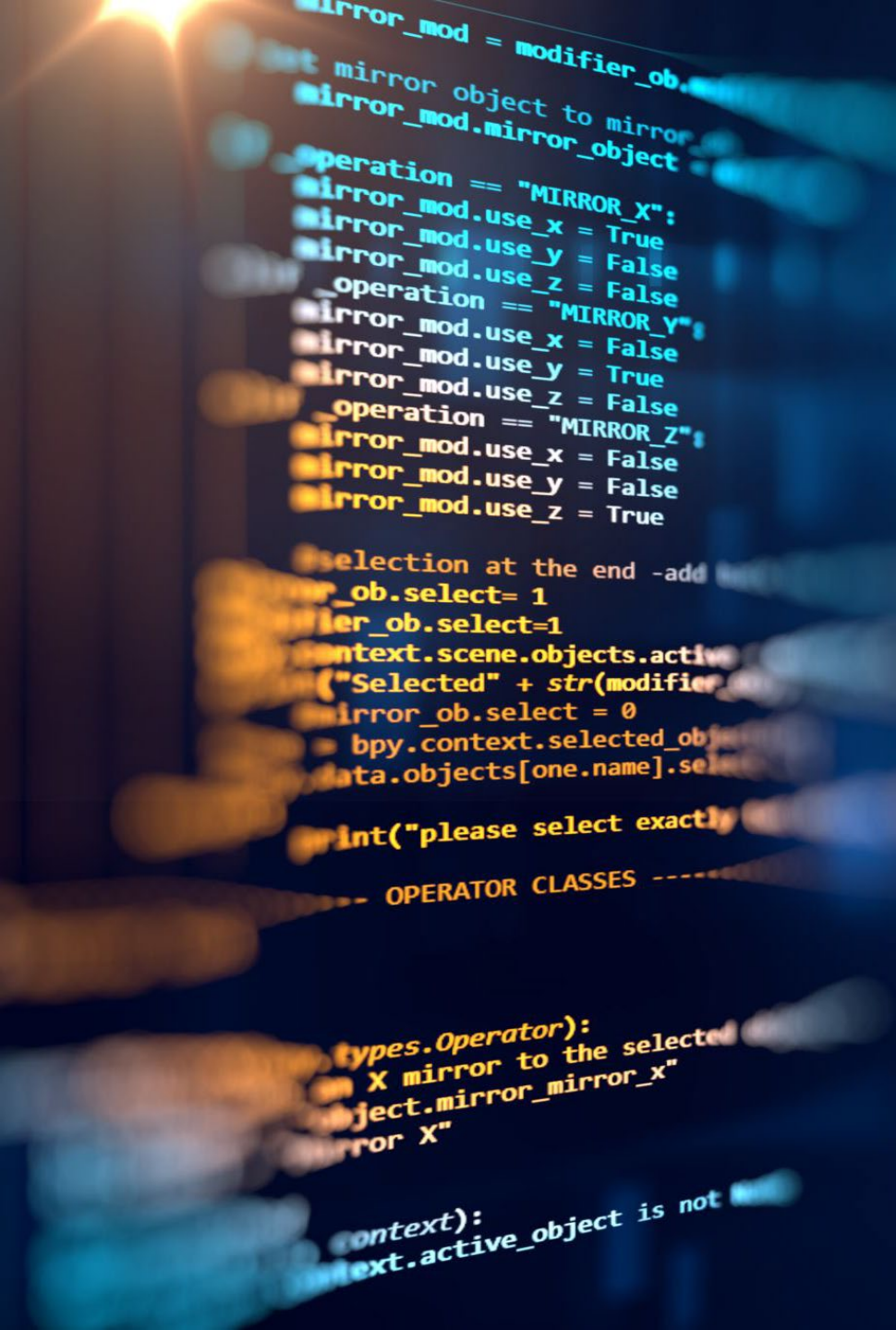


Infrastructure as a service (IaaS)

IaaS use cases

IaaS makes sense in a number of situations:

- When **demand** for computing resources is **volatile**. For example, e-commerce sites experience the most demand during holiday seasons.
- For new **organizations that do not have the capital** to invest in infrastructure on site.
- When organizations need to **grow their IT resources rapidly** (for example, internet startup companies).
- For **temporary projects** or temporary infrastructural needs (when organizations require a large amount of compute power for a limited amount of time).



Infrastructure as a service (IaaS)

IaaS use cases

IaaS *may not be the best option* when:

- Regulatory compliance does not allow data to be offshored or outsourced.
- Applications have strict quality-of-service (QoS) requirements.
- Organizations have existing in-house customized infrastructure to meet their IT needs.

```
mirror_mod = modifier_ob.  
#set mirror object to mirror  
mirror_mod.mirror_object
```

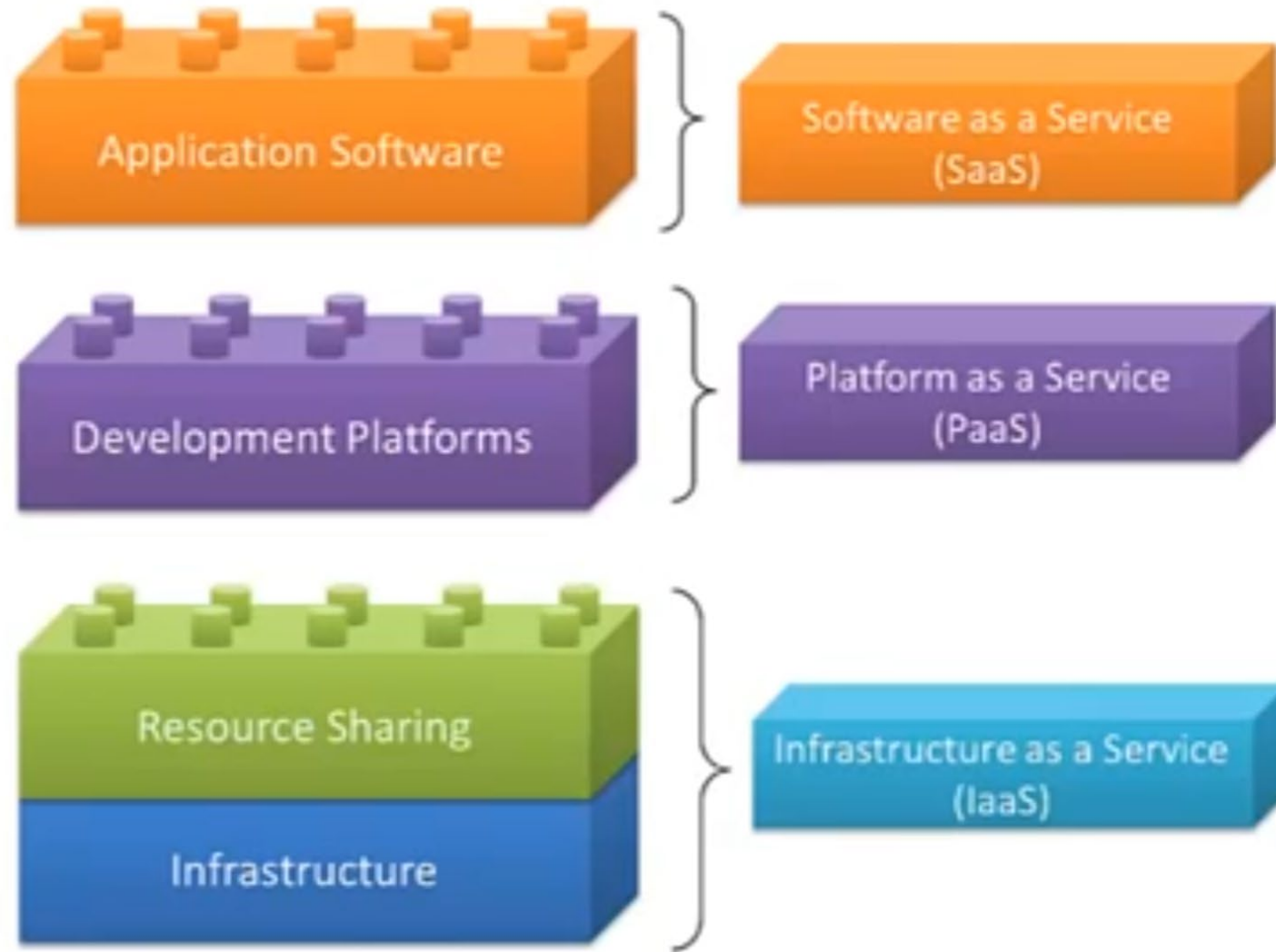
```
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

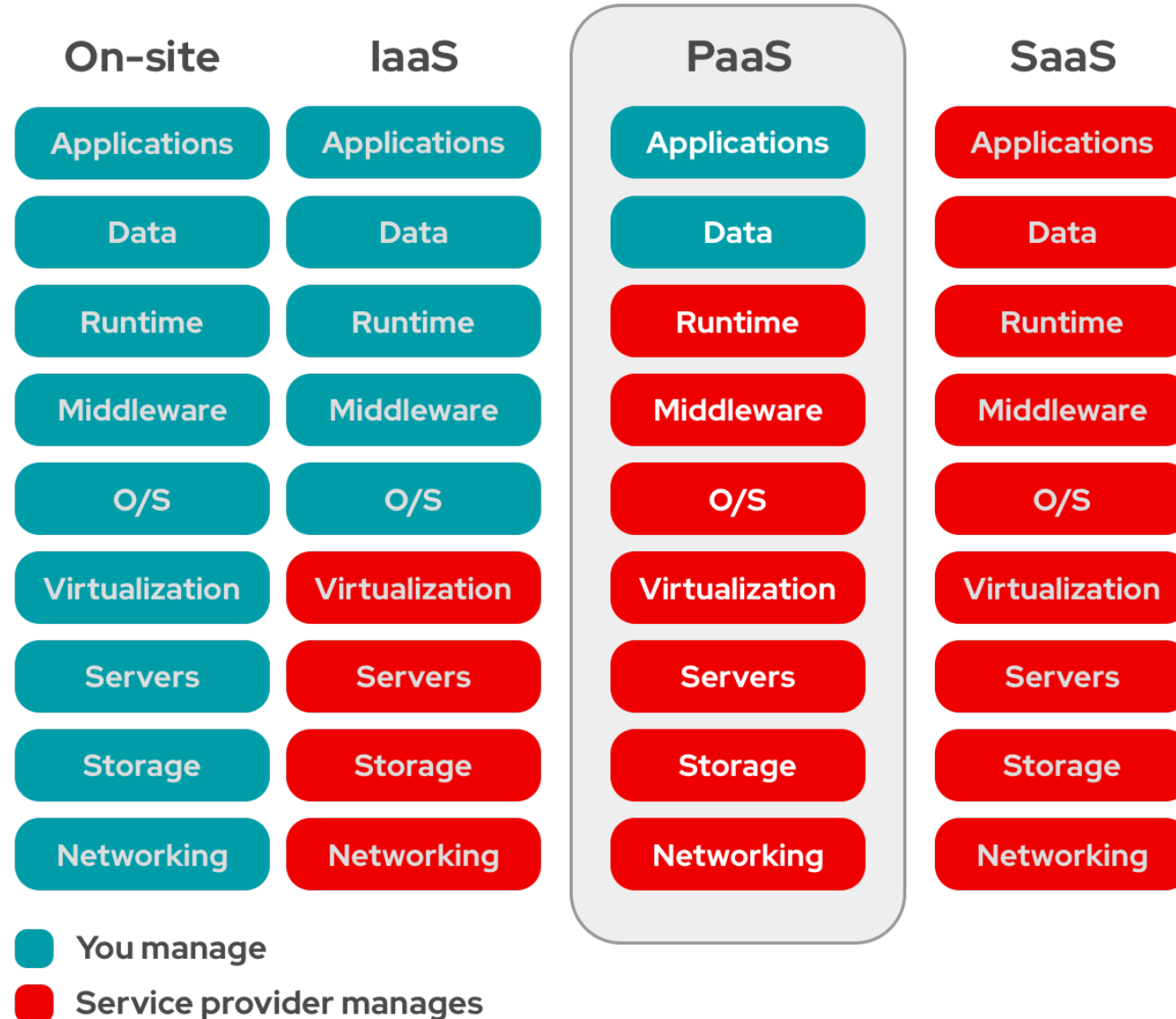
```
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly
```

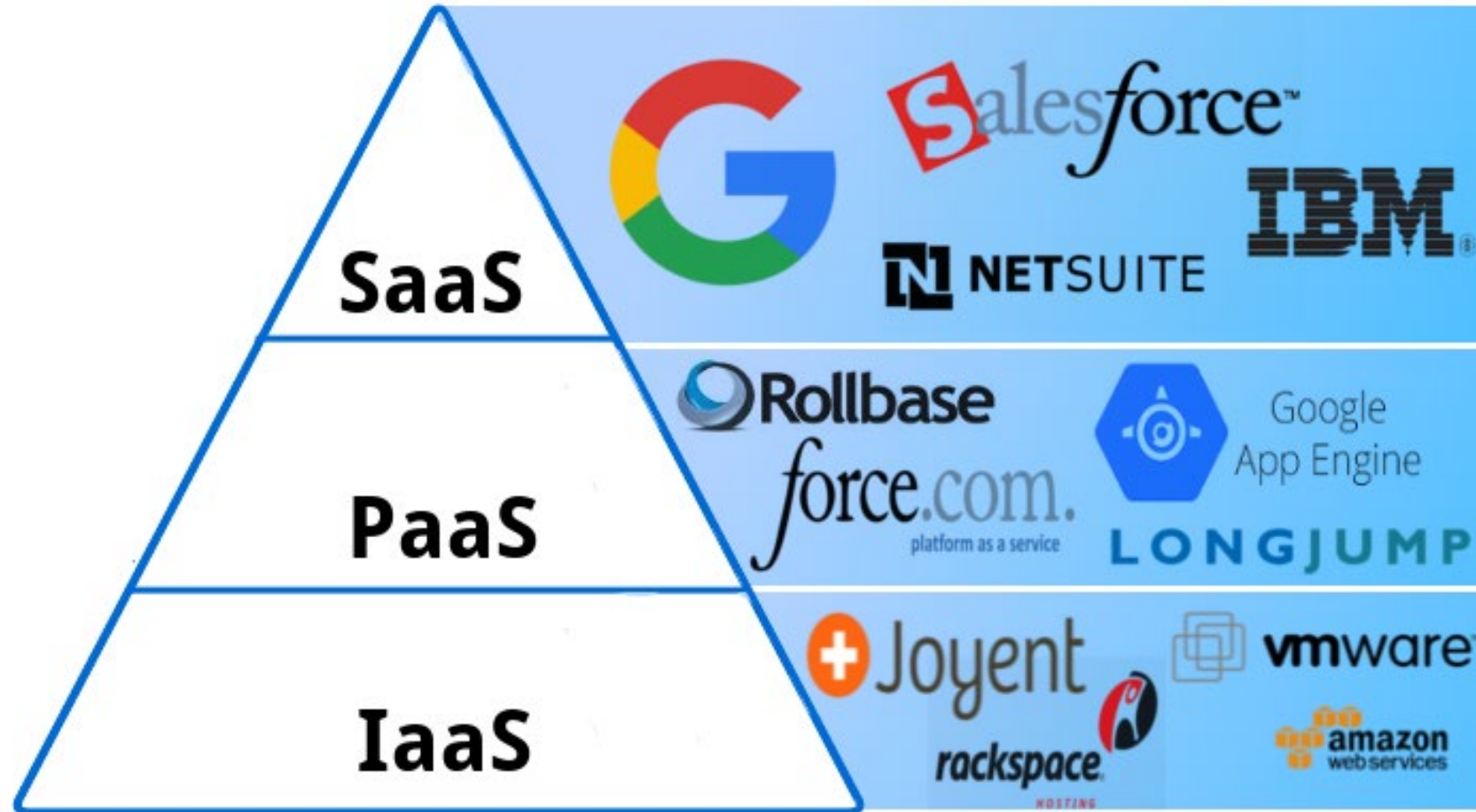
```
--- OPERATOR CLASSES ---
```

```
types.Operator):  
on X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not
```







Any Questions?